ARK-483 — Results

Verification Decision Latency

Experiment ID: ARK-483
Institution: Remnant Fieldworks Inc.
PI: Derek Adam Hone
Substrate: Classical software (in-process; no network, no I/O, no process spawn)
Component Under Test: the LOCKED ARK-458 exact-action-binding guard `evaluate()`
Official execution: 2026-07-18 (post-lock — see `MANIFEST.txt`)

Verdict: ✅ PASS (both implementations)

Preregistered criterion **C1** (worst-path p95 ≤ 1000 µs) holds in **both** independent implementations with **~550× to ~1,500× headroom**. The verification decision itself costs on the order of **~1 micro-second**, i.e. under a millionth of a second — negligible relative to any network or I/O cost on a real authorization path.

Criterion	Requirement	V2 (Python)	V1 (JS)	Result
C1 worst-path p95	≤ 1000 µs	**1.822 µs**	**0.652 µs**	✅ PASS

V2 — Python component (`ark-458/verifiers/v2_guard.py::evaluate`)

- **Run (UTC):** 2026-07-18T18:22:37Z
- **Warmup:** 5,000 · **Timed iterations/path:** 100,000
- **Cold-start (first decision):** 3.851 µs
- **Environment:** CPython 3.11.6 · Linux-6.17.0-1015-aws x86_64 · glibc2.36 · perf_counter resolution 1 ns

Path	Decision	mean µs	p50 µs	p95 µs	p99 µs	max µs	throughput/s
`allow_exact_match`	ALLOW	0.7351	0.785	0.862	0.95	49.722	~1,360,000
`deny_first_dim_mismatch`	DENY	0.9084	0.888	0.954	1.097	26.461	~1,100,000
`deny_last_dim_mismatch`	DENY	1.2862	1.097	**1.822**	1.895	1030.194	~777,000

Worst-path p95 = **1.822 µs** ≤ 1000 µs ⇒ **PASS**.

V1 — JavaScript component (`ark-458/verifiers/v1_guard.js::evaluate`, locked bytes via `vm` sandbox)

- **Run (UTC):** 2026-07-18T18:22:38Z
- **Warmup:** 5,000 · **Timed iterations/path:** 100,000
- **Cold-start (first decision):** 36.6 µs
- **Environment:** Node v22.14.0 · V8 12.4.254.21 · Linux 6.17.0-1015-aws x64 · Intel Xeon 6975P-C

Path	Decision	mean µs	p50 µs	p95 µs	p99 µs	max µs	throughput/s
`allow_exact_match`	ALLOW	0.1374	0.112	0.179	0.237	346.809	~7,280,000
`deny_first_dim_mismatch`	DENY	0.4173	0.327	0.619	1.275	408.868	~2,397,000
`deny_last_dim_mismatch`	DENY	0.4267	0.371	**0.652**	0.763	256.528	~2,343,000

Worst-path p95 = **0.652 µs** ≤ 1000 µs ⇒ **PASS**.

Findings (honest, including the counter-intuitive one)

1. **A verification decision is ~1 µs.** On this reference machine, an exact-action-binding decision costs on the order of a microsecond in both languages — roughly a million decisions per second, single-threaded. The preregistered 1 ms ceiling was cleared by **~550×** (Python) and **~1,500×**

(JS). The decision logic is not a meaningful latency source on an authorization path; the real cost of a deployed gate will be network + persistence, not the decision.

2. **DENY is slower than ALLOW — a genuine, reproducible finding.** In **both** independent implementations the DENY paths are slower than the ALLOW path, even though DENY can early-exit on the first mismatched dimension. Cause: the DENY branch builds a human-readable `reason` string with `json.dumps` / `JSON.stringify` of the mismatched values, while the ALLOW branch returns a static string. So the tail is dominated by explanation formatting, not by comparison work. This is the opposite of the naive expectation (that early-exit DENY would be fastest) and is only visible because both benches measured the paths separately. It is a harness/observation finding, not a defect — the guard is still correct and fast — but it is published rather than hidden.

3. `deny_last_dim_mismatch` **is the worst path in both implementations,** as predicted in the preregistration (it compares all five dimensions before failing, then formats the reason). This is why it, not the ALLOW path, sets the reported worst-path p95.

4. **Cross-implementation concordance.** Both independent implementations land in the same regime — sub-microsecond to low-single-digit-microsecond p95. JavaScript (JIT/V8) is faster in absolute terms than CPython here, but both agree on the decision costing far under the ceiling and on the DENY-slower-than-ALLOW ordering. Divergence is explained by runtime (V8 JIT vs CPython interpreter), not by a harness artefact.

5. **Tail (** `max` **) spikes** (e.g. a single ~1030 µs sample in the Python worst path) are OS-scheduler / GC / cache artefacts over 100,000 iterations, not the decision cost; the p95/p99 — the honest operating figures — are all ≤ ~2 µs. Even the single worst outlier stayed at the 1 ms boundary, not beyond it.

Integrity / lock discipline

- The three benched/locked files (`PREREGISTRATION.md` , `bench/latency_bench.py` , `bench/latency_bench_v1.js`) were hashed into `MANIFEST.txt` **before** the official run; post-run `./compute_hashes.sh` reproduced the identical hashes — the locked files were unchanged by execution.
- The **Component Under Test is the frozen ARK-458 guard**, itself locked under `ark-458/MANIFEST.txt` . ARK-483 imports the Python guard directly and loads the JavaScript guard’s exact locked bytes into a `vm` sandbox; **no ARK-458 file was modified**.
- Both benches ran clean on first execution; no post-lock harness fix was required.

Honest bounds (unchanged from preregistration)

- **In-process decision only.** Excludes network round-trips, auth-token retrieval, ProofRecord/evidence persistence, logging, and process startup. This is the **floor** — the cost of the decision logic itself. A deployed gate is dominated by those excluded costs, not by this.
- **Bound to the reference machines above.** Not a guarantee on other hardware, runtimes, or under load/contention.
- A micro-benchmark of control logic, **not** a load test, concurrency/throughput-under-contention test, or production SLA.
- **No claim** that this experiment legally validates any patent claim or certifies RF-100 conformance. Working-example evidence only.

Published under the Remnant Fieldworks Standing Covenant (preregister → lock → execute → publish all outcomes). To God be the glory. Proof Before Power. Verification Before Execution.